KMP Questions 14–25

Questions and Answers • Kotlin Multiplatform

Question 14

Why is dependsOn not the same as dependencies {}? What are the consequences of confusing them?

Answer: **dependsOn** connects Kotlin source sets and allows them to share code. **dependencies {}** declares external libraries or project dependencies.

Example:
```
jvmMain.dependsOn(commonMain)
commonMain.dependencies {
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3") }
```

Confusing them can cause compilation errors, unresolved references, incorrect source-set relationships, and missing dependencies. Use dependsOn for source-set relationships and dependencies {} for libraries.

Question 15

Why is commonTest.dependsOn(commonMain) incorrect, and why is calling dependsOn directly on a library's commonMain also a mistake?

Answer: The intended relationship is that `commonTest` depends on `commonMain`, not the other way around. Production code should not depend on test code.

Example:
```
commonTest.dependencies { implementation(kotlin("test")) }
```

A consumer normally cannot modify a published library's internal source-set hierarchy. To use that library, declare it as a Gradle dependency.

Question 16

How can a project targeting JVM, JS, and Linux x64 share code between JVM and JS without exposing it to Linux?

Answer: Create an intermediate source set called `jvmAndJsMain`. Make both `jvmMain` and `jsMain` depend on it, but do not connect `linuxX64Main` to it.

Example configuration (adapt to the project's Kotlin Gradle plugin version):
```
val jvmAndJsMain by creating { dependsOn(commonMain.get()) }
jvmMain.get().dependsOn(jvmAndJsMain)
jsMain.get().dependsOn(jvmAndJsMain)
```

Code in jvmAndJsMain is available to JVM and JS, but Linux does not inherit it through this relationship.

Question 17

How can you verify that the default hierarchy template creates jvmAndJsMain automatically?

Answer: Configure JVM and JS targets, then inspect the source sets in IntelliJ IDEA or Android Studio. Create a function in `jvmAndJsMain`, use it from both `jvmMain` and `jsMain`, and compile both targets. If both can access it, the source set works correctly. Modern Kotlin Gradle plugin versions usually configure the default hierarchy automatically.

Question 18

If kotlinx-coroutines-core:1.7.3 is declared in commonMain, must it also be declared separately in androidMain and iosMain?

Answer: No. A dependency declared in commonMain is available to platform compilations that include it through the source-set hierarchy.

Example:

```
commonMain.dependencies {
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3") }
```

Android and iOS can use the shared coroutine APIs without declaring the same dependency again. Platform-specific libraries should be declared in their respective platform source sets.

Question 19

Why might Dispatchers.Main and asExecutor() behave differently when compiling common and platform-specific code?

Answer: Kotlin Multiplatform supports different APIs on different platforms. Dispatchers.Main depends on a platform's main-dispatcher implementation, while asExecutor() is a JVM-specific interoperability API. Therefore, asExecutor() should not be used directly in commonMain because Java's Executor API is not available on all targets. Keep portable coroutine logic in commonMain and move JVM-specific code into a JVM or Android source set.

Question 20

If commonMain declares coroutines version 1.7.3 but an Android dependency requires a newer version, what happens? Why might upgrading only the Android dependency be insufficient?

Answer: Gradle resolves dependency versions for relevant configurations using the dependency graph, constraints, and resolution rules. Android may resolve a newer version, but changing an Android-only dependency does not guarantee commonMain can compile against the required API. Align dependency versions explicitly and ensure the selected version is compatible with the project's Kotlin version and other dependencies.

Question 21

How can you print JVM compilation names, and what is the difference between a source set and a compilation?

Answer: A source set groups Kotlin source files, dependencies, and language settings. A compilation combines relevant source sets to produce output for a target and purpose.

Example:

```
kotlin { jvm { compilations.all { println("Compilation: $name") } } }
```

For a JVM target, compilation names typically include `main` and `test`. Exact output depends on project configuration and plugin version.

Question 22

What are the advantages and disadvantages of one shared module compared with several shared modules? When should a team migrate?

Answer: One shared module is easier to configure, has simpler dependency management, and suits small projects, but may become large and tightly coupled. Multiple shared modules improve separation of responsibilities, modularity, and independent team ownership, but require more configuration and dependency management. Split a module when it becomes hard to maintain or applications need different subsets of functionality.

Question 23

Why can generating a separate iOS framework for each feature module cause problems, and how can an umbrella module help?

Answer: If feature frameworks each embed their own copy of a shared core dependency, the iOS application may contain duplicated code or separate instances of shared state. For example, authentication and profile modules might access different copies of a singleton. An umbrella module depends on feature modules and exposes required APIs through a consolidated framework. This can reduce duplication, although the dependency graph and framework configuration must still be checked.

Question 24

What are the advantages and disadvantages of a monorepo, a two-repository split, and a three-repository split? What is the versioning consequence?

Answer: Monorepo: all applications and shared code are together, making coordinated changes easier, but the repository and build can become complex. Two repositories: Android and shared code are together while iOS is separate, balancing flexibility with some coordination overhead. Three repositories: Android, iOS, and shared code are separate, enabling independent releases but requiring explicit shared-library distribution and versioning. Applications must consume compatible shared-library versions.

Question 25

How should a company with five applications design shared logging, networking, and analytics libraries when not every application needs every library?

Answer: Create separate modules for logging, networking, and analytics. Each application should declare only the modules it needs.

Example:

```
commonMain.dependencies { implementation("com.company:logging:1.2.0");
implementation("com.company:networking:2.1.0") }
```

Use local modules when applications are developed together in one repository. Use remote, versioned libraries when teams work in separate repositories and need independent releases. This reduces unnecessary dependencies and allows applications to upgrade shared libraries independently.

Revision Summary

- 14 — dependsOn vs dependencies {}
- 15 — Correct source-set dependency direction
- 16 — Sharing code between selected targets
- 17 — Default hierarchy template

- 18 — Dependency propagation
- 19 — Common APIs vs platform-specific APIs
- 20 — Gradle dependency version resolution
- 21 — Source sets vs compilations
- 22 — Single shared module vs multiple modules
- 23 — iOS framework duplication and umbrella modules
- 24 — Monorepo vs two-repository vs three-repository setup
- 25 — Modular shared libraries and dependency management